How Different Kernels Change the Behaviour of Support Vector Machines (SVMs): A Practical Tutorial

Name:

ID:

Module: Machine Learning

Date: DATE

GitHub Repository: (https://github.com/Abbuzaid/svm-kernel-tutorial)

Table of Contents

Abstract 3
1. Introduction (why kernels matter) 4
2. Step-by-step practical workflow 4
Step 1 Import libraries 4
Explanation: 4
Step 2 Load dataset and select features for 2D plots 4
Explanation: 5
Step 3 Train/test split and standardisation 5
Explanation: 5
Step 4 Train SVMs with different kernels 5
Explanation: 5
Step 5 Evaluate accuracy (test + cross-validation) 5
Explanation: 6
Step 6 Confusion matrix (per-class performance) 6
Explanation: 6
3. Visual Results (figures from notebook) 6
4. Hyperparameters and behaviour (short, practical) 8
4.1 $\boldsymbol{C}$ (regularisation strength) 8
4.2 $\boldsymbol{\gamma}$ (RBF locality) 8
4.3 Polynomial degree 9
5. Practical recommendations (what to do in real work) 9
6. Ethical considerations (brief) 9
7. Conclusion 9
References 10

Table of Figures:

Figure 1:dataset_overview.png class distribution in feature space 6
Figure 2:kernel_comparison.png decision boundaries for each kernel 7
Figure 3:accuracy_comparison.png train vs test accuracy + support vectors 7
Figure 4:confusion_matrices.png confusion matrices per kernel 7
Figure 5: c_parameter_effect.png effect of C (RBF) 7

Figure 6: gamma_effect.png effect of γ (RBF) 8
Figure 7:: degree_effect.png effect of degree (Polynomial) 8

Abstract

This tutorial demonstrates how kernel choice changes the decision boundary and performance of a Support Vector Machine (SVM). Using the Wine dataset (3 classes) and two features for visualisation, we train Linear, RBF, Polynomial and Sigmoid SVMs, compare accuracy and confusion matrices, and show how hyperparameters C, γ and polynomial degree affect underfitting and overfitting. Code snippets are provided so the reader can reproduce the results in Python using scikit-learn. Kernel methods remain important in practical machine learning because they enable non-linear classification while retaining strong theoretical foundations and competitive performance on structured tabular datasets (Pedregosa et al., 2018; Raschka, Liu, Mirjalili, & Dzhulgakov, 2022).

1. Introduction (why kernels matter)

Support Vector Machines are supervised learning models used for classification and regression. A linear SVM separates classes using a maximum-margin hyperplane. However, many real datasets are not linearly separable, so SVMs use kernel functions to implicitly map data into higher-dimensional feature spaces where separation becomes easier. In practice, kernel selection and hyperparameter tuning strongly affect both predictive performance and the shape of the decision boundary (Raschka et al., 2022; Scikit-learn Developers, 2024).

Kernels matter because they control how similarity between data points is measured. A linear kernel produces straight decision boundaries, while non-linear kernels such as Radial Basis Function (RBF) and Polynomial can create more flexible surfaces. This flexibility can improve classification when classes overlap in a non-linear way, but it also increases the risk of overfitting if hyperparameters are poorly chosen (Buitinck et al., 2018; Scikit-learn Developers, 2024).

2. Step-by-step practical workflow

Step 1 Import libraries

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn import datasets

from sklearn.svm import SVC

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

Explanation:

We use scikit-learn's SVC implementation, together with scaling and evaluation utilities. Scikit-learn is a widely used Python machine learning library for implementing classification workflows reproducibly (Pedregosa et al., 2018; Scikit-learn Developers, 2024).

Step 2 Load dataset and select features for 2D plots

```
Python

wine = datasets.load_wine()

X = wine.data[:, :2]     # alcohol, malic_acid (2D visualisation)

y = wine.target
```

Explanation:

The Wine dataset contains 178 samples and 3 classes. We use only the first two features so that decision boundaries can be plotted in two dimensions. This is a tutorial-oriented simplification; using all 13 features would likely improve predictive performance, but would not allow direct 2D visualisation (Scikit-learn Developers, 2024).

Step 3 Train/test split and standardisation

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

Explanation:

Scaling is important because SVMs are sensitive to feature magnitudes, especially when kernels depend on distances, such as RBF and Polynomial. Standardisation ensures that each feature contributes more evenly to the model and usually improves optimisation and performance (Raschka et al., 2022; Scikit-learn Developers, 2024).

Step 4 Train SVMs with different kernels

```
kernels = ["linear", "rbf", "poly", "sigmoid"]
models = {}
for k in kernels:
    clf = SVC(kernel=k, C=1.0, gamma="scale", degree=3, random_state=42)
    clf.fit(X_train_s, y_train)
    models[k] = clf
```

Explanation:

We keep most parameters fixed so differences come mainly from the choice of kernel. The degree parameter only affects the Polynomial kernel, while gamma is particularly important for RBF, Polynomial and Sigmoid kernels (Scikit-learn Developers, 2024).

Step 5 Evaluate accuracy (test + cross-validation)

```
for k, clf in models.items():
    test_acc = clf.score(X_test_s, y_test)
    cv = cross_val_score(clf, X_train_s, y_train, cv=5).mean()
    print(k, "test:", round(test_acc, 3), "cv:", round(cv, 3))
```

Explanation:

Test accuracy estimates performance on unseen data, while cross-validation provides a more stable estimate than a single train/test split. Cross-validation is especially useful when model behaviour changes substantially across different kernel and hyperparameter choices (Raschka et al., 2022).

Step 6 Confusion matrix (per-class performance)

```
k = "rbf"

y_pred = models[k].predict(X_test_s)

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(cm, display_labels=wine.target_names)

disp.plot(cmap="Blues", colorbar=False)

plt.title("Confusion Matrix (RBF Kernel)")

plt.show()
```

Explanation:

Confusion matrices show which classes are predicted correctly and which are confused. This is useful in multiclass classification because a single accuracy score can hide class-specific weaknesses (Scikit-learn Developers, 2024).

3. Visual Results (figures from notebook)

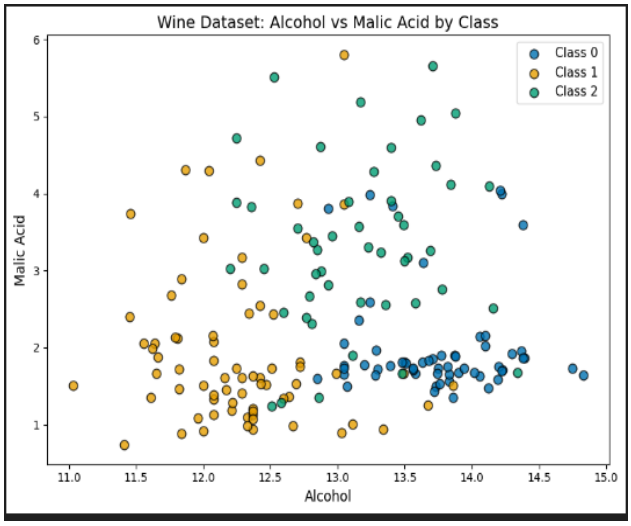

Figure 1:dataset_overview.png — class distribution in feature space

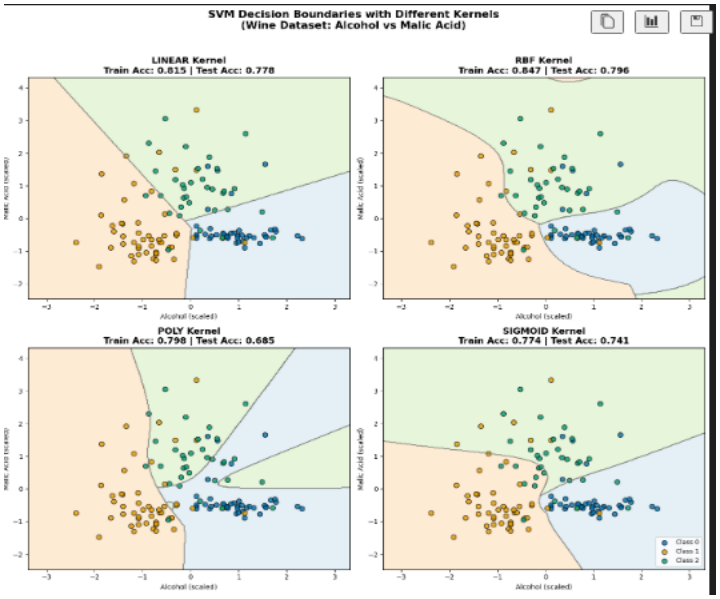

Figure 2:kernel_comparison.png decision boundaries for each kernel

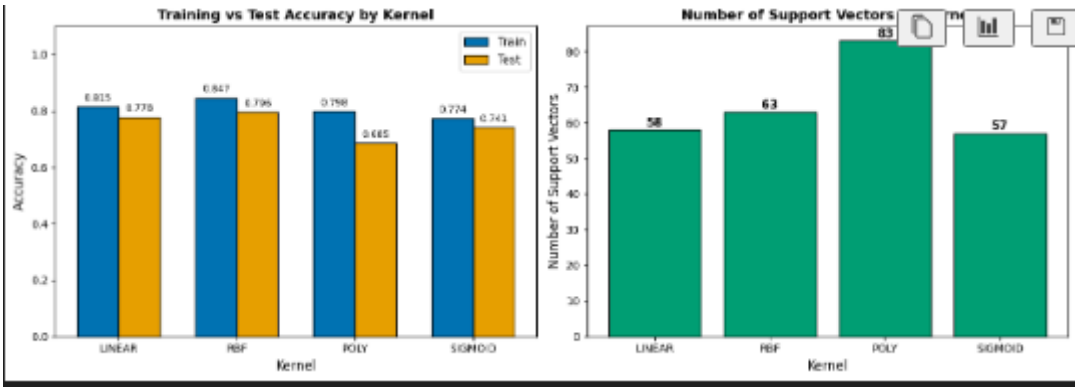

Figure 3:accuracy_comparison.png train vs test accuracy + support vectors

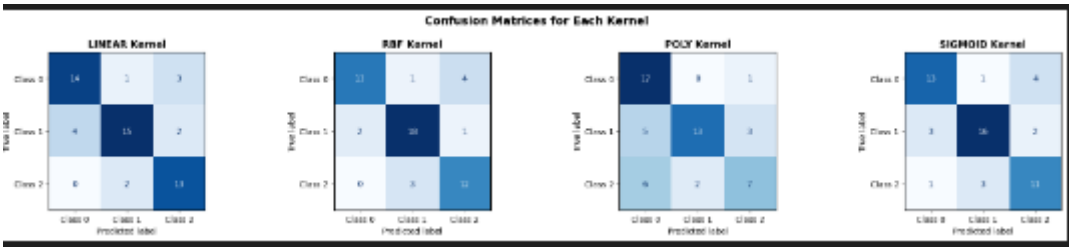

Figure 4:confusion_matrices.png confusion matrices per kernel

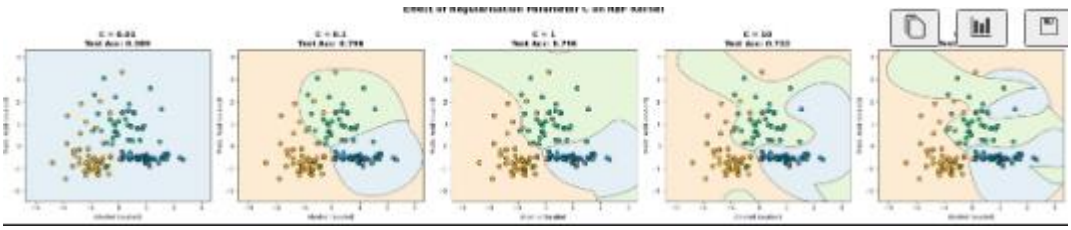

Figure 5: c_parameter_effect.png effect of C (RBF)

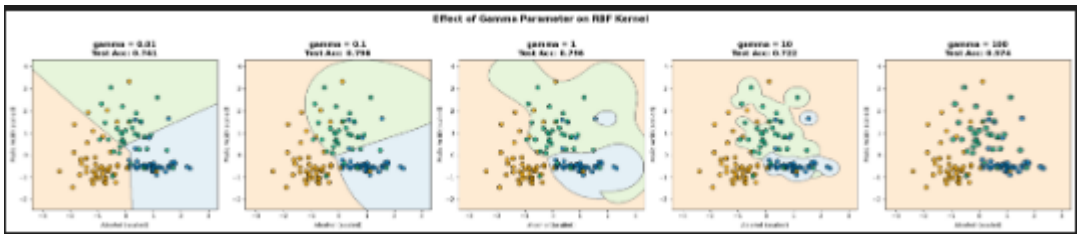

Figure 6: gamma_effect.png effect of γ (RBF)

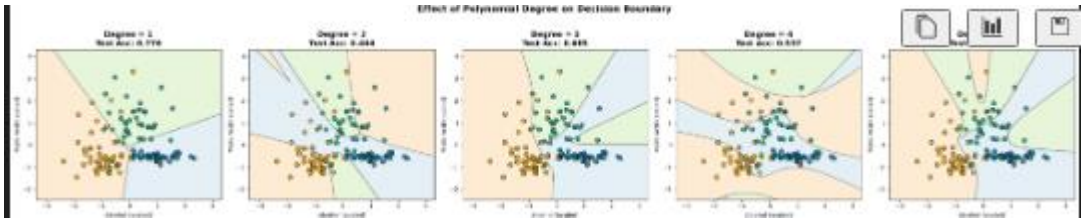

Figure 7:: degree_effect.png effect of degree (Polynomial)

What to say (briefly):

Linear: straight boundaries; may underfit if classes overlap non-linearly**.**

RBF: flexible curved boundaries; usually a strong default when the true structure is unknown.

Polynomial: can capture feature interactions; complexity increases with degree.

Sigmoid: often less stable and may underperform in standard classification tasks; included mainly for comparison (Scikit-learn Developers, 2024; Raschka et al., 2022).

4. Hyperparameters and behaviour (short, practical)

4.1 C (regularisation strength)

Small C: allows more margin violations, producing a simpler boundary and increasing underfitting risk.

Large C: penalises classification errors more strongly, producing a more complex boundary and increasing overfitting risk.

This reflects the central SVM trade-off between margin maximisation and training error minimisation (Raschka et al., 2022).

4.2 γ (RBF locality)

Small γ: each point has a wider area of influence, resulting in smoother decision boundaries and possible underfitting.

Large γ: each point influences only a small local region, creating more irregular boundaries and possible overfitting.

In practice, γ strongly controls how locally sensitive the classifier becomes (Scikit-learn Developers, 2024).

4.3 Polynomial degree

Degree 1 is approximately linear. Higher degree allows more complex curved boundaries. Very high degree may overfit and increase computational cost. Polynomial kernels can be useful when interactions between features are meaningful, but they are often less robust than RBF without careful tuning (Raschka et al., 2022).

5. Practical recommendations (what to do in real work)

Always scale features before training an SVM. Start with **RBF** if there is no strong reason to prefer another kernel. Use cross-validation or grid search to tune C and γ.

Use a **linear SVM** for high-dimensional problems, where it is often faster and provides a strong baseline. Prefer simpler models when performance is similar, because simpler models often generalise better and are easier to justify in practice (Scikit-learn Developers, 2024; Raschka et al., 2022).

6. Ethical considerations (brief)

SVMs can reflect biases present in training data and may be difficult to explain to non-technical stakeholders, especially when non-linear kernels are used. In high-stakes contexts, model selection should consider not only accuracy but also fairness, transparency and reproducibility. Interpretability tools and clear documentation can help make non-linear models more accountable in applied settings (Molnar, 2022; NIST, 2023).

7. Conclusion

This tutorial showed that kernel choice changes both SVM decision boundaries and classification performance. RBF and Polynomial kernels can model non-linear class separation, while Linear SVMs may underfit when classes overlap in curved ways. Hyperparameters such as C, γ and degree directly control model complexity and must be tuned carefully to avoid underfitting or overfitting. Overall, the RBF kernel is often a strong practical default, while Linear SVMs remain useful baselines for simpler or high-dimensional tasks. The provided code snippets and notebook figures support full reproducibility of the analysis (Scikit-learn Developers, 2024; Raschka et al., 2022).

References

Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., ... Varoquaux, G. (2018). API design for machine learning software: Experiences from the scikit-learn project

Molnar, C. (2022). *Interpretable machine learning* (2nd ed)

NIST. (2023). *AI Risk Management Framework (AI RMF 1.0)*. National Institute of Standards and Technology.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2018). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*.

Raschka, S., Liu, Y., Mirjalili, V., & Dzhulgakov, D. (2022). *Machine learning with PyTorch and scikit-learn*. Packt.

Scikit-learn Developers. (2024). *Support Vector Machines*. Scikit-learn documentation.